

Unconstrained Optimisation

Saurav Samantaray

Department of Mathematics

Indian Institute of Technology Madras

September 30, 2026



Unconstrained Optimisation

- Minimise an objective function that depends on real variables.
- No restriction on the values of these variables (no constraints).

Mathematical Formulation:

$$\begin{aligned} \min_x f(x) \\ \text{where, } x \in \mathbb{R}^n, n \geq 1. \\ f : \mathbb{R}^n \rightarrow \mathbb{R} \text{ is smooth} \end{aligned} \tag{1}$$

In a real world scenario

- The objective function " f " might not be known globally everywhere.
- Ideally, may have finitely many values of " f " or some derivatives of " f ".
- Any information for " f " at arbitrary points usually do-not come very cheaply.
- Therefore, one should prefer for algorithms which do-not demand the same, unnecessarily.

Example

- Suppose we are trying to find a curve that fits some experimental data.
- (t_i, y_i) , y_i signal is measured at time t_i .
- Let's assume based on the knowledge of the phenomenon under study we have the understanding that the signal has exponential and oscillatory behaviour of certain types.

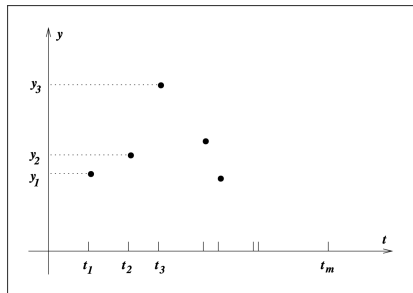


Figure: Least squares data fitting problem.

Example

- Choose the model function as

$$\phi(t, x) = x_1 + x_2 e^{-(x_3 - t)^2 / x_4} + x_5 \cos(x_6 t)$$

where x_i 's are the parameters of the model.

- What we want is the model should fit the observed data y_j , as closely as possible.
- Let $x = (x_1, x_2, x_3, x_4, x_5, x_6)$,
We define the residual for each y_j as

$$r_j = y_j - \phi(t_j, x), \quad j = 1, \dots, m.$$

- We define the objective function as

$$\min_{x \in \mathbb{R}^6} f(x) = r_1^2(x) + \dots + r_m^2(x)$$

This is a non-linear least square problem, a special case of unconstrained optimisation.

Example

- Note that the equation of the objective function appears quite expensive even for small number of variables

$$n = 6$$

- say, if the no. of measurements i.e. $m = 10^5$, then the evaluation of f becomes quite a computational expense.

Lets Gain Some Perspective!!

- Suppose for a given set of data the optimal solution to the previous problem is approximately

$$x^* = (1.1, 0.01, 1.2, 1.5, 2.0, 1.5)$$

and the corresponding function value is $f(x^*) = 0.34$.

- As at the optimal point the objective is non-zero there must be some discrepancy between the function values and the observations made.

Some Perspective

- i.e. y_j and $\phi(t_j, x^*)$ aren't the same for some or many
 $(y_j, t_j) \longleftrightarrow \phi(t_j, x^*)$
- The model hasn't produced all the data points correctly as

$$f(x^*) \neq 0$$

- Then how to know x^* is indeed a minimiser of f ?
- In the sense that how to know which all points one should go close to or not?
- To answer this question, we need to define the term "solution" and explain how to recognise solutions.

What is a solution?

A point x^* is a global minimiser of f if

$$f(x^*) \leq f(x) \quad \forall x \in \mathbb{R}^n$$

or in the domain of interest.

- It would be the most ideal scenario if we could find a global minimiser.
- It might be difficult to get a global minimiser, owing to the limited (or local) knowledge of f .
- Most algorithms are only able to find a local minimiser.

What is a solution?

Local Minimiser

A point x^* is called a local minimiser, if there is a neighbourhood $\mathcal{N}$ of x^* such that

$$f(x^*) \leq f(x) \quad \forall x \in \mathcal{N}$$

- It's a points that achieves the smallest value of f in its neighbourhood.

Weak Local Minimiser

$$f(x^*) \leq f(x) \quad x \in \mathcal{N}$$

Strict (Strong) Local Minimiser

$$f(x^*) < f(x) \quad x \in \mathcal{N}, x \neq x^*$$

Example

- For a constant function $f(x) = 2$ every point is a weak local minimiser.
- For $f(x) = (x - 2)^4$, $x = 2$ is a strict local minimiser.

Isolated Local Minimiser

A point x^* is called an isolated local minimiser if there is a neighbourhood $\mathcal{N}$ of x^* such that x^* is the only local minimiser in $\mathcal{N}$.

Example

$$f(x) = x^4 \cos(1/x) + 2x^4 \quad f(0) = 0$$

- is twice continuously differentiable
- has a strict local minimiser at $x^* = 0$
- however, there are strict local minimisers at many nearby points x_j ,
and $x_j \rightarrow 0$ as $j \rightarrow \infty$

A Zoom plot of $f(x)$ around $x = 0$

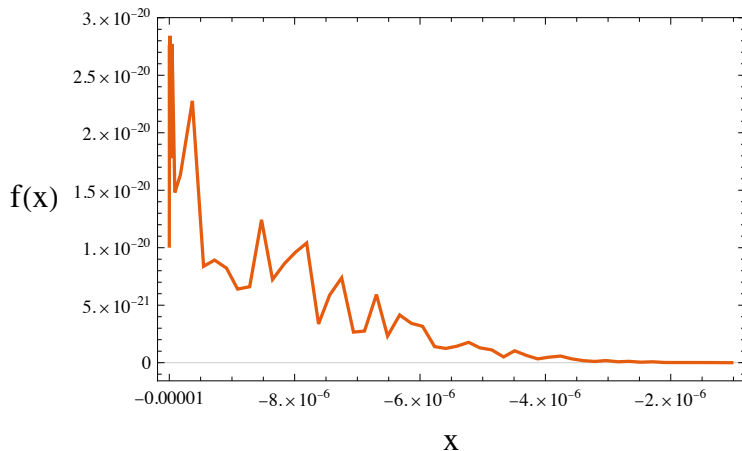


Figure: Showcases many strict local minimisers near $x = 0$.

- Some strict local minimisers are not isolated
- All isolated local minimisers are strict
- It is often difficult to determine a global minimiser for an algorithm, as it often gets trapped in a locality (at a local minimiser)

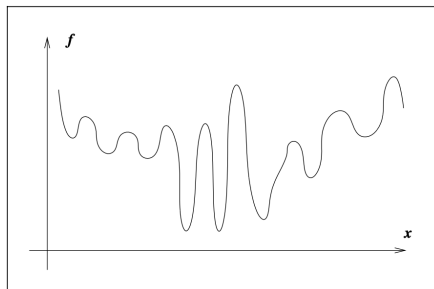


Figure: Showcases a function with many local minimisers.

How to detect minimisers?

- The simplest test being

$$f'(x^*) = 0$$

is very insufficient to speak anything about the globality of the minimiser.

- These cases (having a lot of local minimisers) is quite standard for optimisation problems.
- Global knowledge about a function f may help identify global minima.
- For convex functions local minimiser is also a global minimiser.

Recognising A Local Minima

- It seems the only way to conclude a point is a local minimum is by comparing the functional values at every point.
- However, if the function f is smooth, more efficient ways can be thought of to identify local minima.

Theorem (First-Order Necessary Conditions):

If x^* is a local minimizer and f is continuously differentiable in an open neighbourhood of x^* , then

$$\nabla f(x^*) = 0.$$

Remark:

Therefore, for any point to be a minimiser of a function it has to be a critical point.

Stationary Point

Definition

A point x^* is called a **stationary** point for f if

$$\nabla f(x^*) = 0.$$

- Any local minimiser must be a stationary point for smooth functions.
- B a matrix is **positive definite** if $p^T B p > 0$ for all vectors $p \neq 0$.
- **positive semi-definite** if $p^T B p \geq 0$ for all p .

Second Order Necessary Conditions

Theorem

If x^* is a local minimiser of f and $\nabla^2 f$ exists and is continuous in an open neighbourhood of x^* , then

$$\nabla f(x^*) = 0 \quad \text{and} \quad \nabla^2 f(x^*) \text{ is positive semi-definite.}$$

Second Order Sufficient Conditions

Theorem

Suppose that $\nabla^2 f$ is continuous in an open neighbourhood of x^ and that $\nabla f(x^*) = 0$ and $\nabla^2 f(x^*)$ is positive definite. Then x^* is a strict local minimiser of f .*

Global Minimiser for Convex Functions

Definition

A function $f : \mathbb{S} \rightarrow \mathbb{R}^n$, $\mathbb{S} \subset \mathbb{R}^n$ is said to be convex, if $\mathbb{S}$ is convex and for any two points x and y in $\mathbb{S}$ the graph of f lies below the straight line connecting $(x, f(x))$ and $(y, f(y))$ in the space $\mathbb{R}^{n+1}$. That is we have the following:

$$f(\alpha x + (1 - \alpha)y) \leq \alpha f(x) + (1 - \alpha)f(y), \text{ for all } \alpha \in [0, 1]$$

- When f is smooth as well as convex and the dimension n is 1 or 2, the graph of f is bowl-shaped and its contours define convex sets.
- A function f is said to be concave if $-f$ is convex.

Theorem

When f is convex, any local minimiser x^ is a global minimiser of f . If in addition f is differentiable, then any stationary point x^* is a global minimiser of f .*

Steepest Descent Method

We want to solve the unconstrained optimization problem

$$\min_{x \in \mathbb{R}^n} f(x).$$

Starting from an initial guess x_0 , we want to generate a sequence

$$x_0, x_1, x_2, \dots$$

that approaches a minimizer x^* .

At each point x_k , we need to answer:

- Which direction should we move?
- How far should we move?

Direction of Steepest Descent

For a small displacement p ,

$$f(x_k + p) \approx f(x_k) + \nabla f(x_k)^T p.$$

Therefore, the change in f is approximately

$$\Delta f \approx \nabla f(x_k)^T p.$$

For a unit direction $\|p\| = 1$, the most negative value is obtained for

$$p = -\frac{\nabla f(x_k)}{\|\nabla f(x_k)\|}.$$

Hence,

$$-\nabla f(x_k)$$

is the direction of **steepest decrease**.

Steepest Descent Iteration

We move in the negative-gradient direction:

$$x_{k+1} = x_k - \alpha \nabla f(x_k)$$

where $\alpha > 0$ is the **step size**.

For this introductory algorithm, we choose a small fixed value of α .

For example,

$$\alpha = 0.01.$$

The iteration therefore consists simply of

Evaluate gradient $\longrightarrow$ Move downhill $\longrightarrow$ Repeat

- Too small α : convergence can be very slow.
- Too large α : the iteration can overshoot or diverge.

Stopping Criterion

At a minimizer x^* ,

$$\nabla f(x^*) = 0.$$

Therefore, we can stop when

$$\|\nabla f(x_k)\| < \text{tol}$$

for a prescribed tolerance.

For example,

$$\text{tol} = 10^{-6}.$$

We should also impose a maximum number of iterations to prevent an infinite loop.

Steepest Descent: Pseudocode

Input:

- f , gradient ∇f
- initial point x_0 , step size α
- tolerance tol , maximum iterations max_iter

Algorithm:

- 1 Set $x = x_0$.
- 2 For $k = 1, 2, \dots, \text{max_iter}$:

- Compute

$$g = \nabla f(x).$$

- If $\|g\| < \text{tol}$, stop.
- Update

$$x = x - \alpha g.$$

- 3 Return x .

The essential update is:

$x \leftarrow x - \alpha \nabla f(x)$

From Steepest Descent to Newton

In steepest descent,

$$x_{k+1} = x_k - \alpha \nabla f(x_k).$$

- The direction is determined only by the gradient.
- But the gradient tells us only the **local slope**.
- Can we also use information about the **curvature** of f ?
- The second-order Taylor approximation gives

$$f(x_k + p) \approx f(x_k) + \nabla f(x_k)^T p + \frac{1}{2} p^T H(x_k) p,$$

where

$$H(x_k) = \nabla^2 f(x_k)$$

is the Hessian.

Minimize the Quadratic Model

Define the local quadratic model

$$m_k(p) = f(x_k) + \nabla f(x_k)^T p + \frac{1}{2} p^T H(x_k) p.$$

- Instead of choosing a direction only from the gradient, choose p to minimize this quadratic model.
- Set its derivative with respect to p to zero:

$$\nabla_p m_k(p) = \nabla f(x_k) + H(x_k)p = 0.$$

Therefore,

$$H(x_k)p = -\nabla f(x_k)$$

The solution p is the **Newton direction**.

Newton Direction

Let

$$g_k = \nabla f(x_k), \quad H_k = \nabla^2 f(x_k).$$

The Newton direction d_k is obtained from

$$H_k d_k = -g_k$$

If H_k is invertible,

$$d_k = -H_k^{-1} g_k.$$

- However, we **do not compute the inverse explicitly**.
- Instead, we solve the linear system

$$H_k d_k = -g_k.$$

This is an important computational connection to **numerical linear algebra**.

Newton Iteration

Once the Newton direction has been obtained,

$$x_{k+1} = x_k + d_k$$

or equivalently,

$$x_{k+1} = x_k - H_k^{-1} g_k$$

Compare with steepest descent:

$$x_{k+1} = x_k - \alpha g_k.$$

The main change is

$$-g_k \longrightarrow d_k : H_k d_k = -g_k$$

Newton's method uses both

gradient + curvature

to determine the step.

Newton Method: Pseudocode

Input:

- f , gradient ∇f , Hessian $\nabla^2 f$
- initial point x_0
- tolerance tol , maximum iterations max_iter

Algorithm:

- 1 Set $x = x_0$.
- 2 For $k = 1, 2, \dots, \text{max_iter}$:
 - Compute $g = \nabla f(x)$.
 - If $\|g\| < \text{tol}$, stop.
 - Compute $H = \nabla^2 f(x)$.
 - Solve

$$Hd = -g.$$

- Update

$$x = x + d.$$

- 3 Return x .